



Cloud Computing Lab 08

Submitted To:

Sir Shoaib & Sir Waqas

Submitted By:

Hamail Fatima

Roll No:

2023-BSE-023

Section:

5(A)

Task 1 — GitHub CLI, Codespace setup and authentication

Goal: Install GH CLI (if not already present), authenticate with GitHub, create a Codespace, and connect to it. All work for the rest of the lab must be done inside the Codespace.

Steps (do these from your local machine shell first, then run codespace commands):

1. (Local desktop) Install GitHub CLI (Windows example via winget):

winget install --id GitHub.cli

- Save screenshot as: task1_gh_install.png — terminal showing the winget install command output.

```
Do you agree to all the source agreements terms?
[Y] Yes [N] No: Y
Found GitHub CLI [GitHub.cli] Version 2.83.2
This application is licensed to you by its owner.
Microsoft is not responsible for, nor does it grant any licenses to, third-party packages.
Downloading https://github.com/cli/cli/releases/download/v2.83.2/gh_2.83.2_windows_amd64.msi
17.7 MB / 17.7 MB
Successfully verified installer hash
Starting package install...
Successfully installed
PS C:\Users\admin> |
```

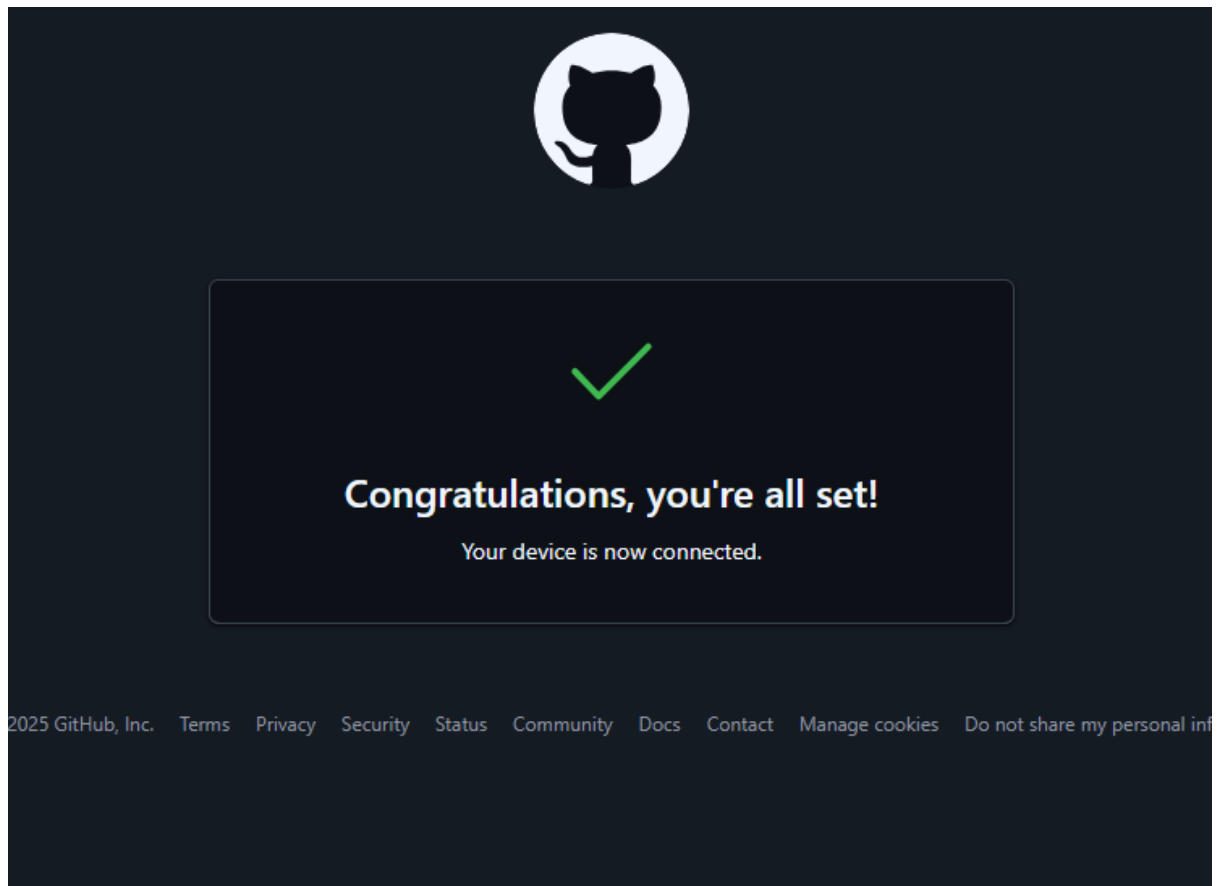
2. (Local) Authenticate GH CLI for Codespaces:

gh auth login -s codespace

- When prompted, generate a GitHub Access Token (classic) in the browser with the following Token scopes:
 - admin:org
 - codespace
 - repo
- Copy the token into the GH CLI prompt to complete login.
- Save screenshot as: task1_gh_auth_login.png — screenshot of gh auth login confirmation (redact token).

```
First copy your one-time code: E2D3-0EA1
Press Enter to open https://github.com/login/device in your browser...
/ Authentication complete.
- gh config set -h github.com git_protocol https
/ Configured git protocol
/ Logged in as Hamail-maker
PS C:\Users\admin> |
```

-



Note: If your organization enforces SSO or device MFA, follow the organization's flow.

3. (Local) List available Codespaces (optional verification):

`gh codespace list`

- Save screenshot as: `task1_codespace_list.png` — `gh codespace list` output (show codespace name).

```
PS C:\Users\admin> gh codespace list
no codespaces found
PS C:\Users\admin> |
```

4. (Local) Create or connect to a Codespace. To create a new codespace from the current repo:

`gh codespace create --repo <owner>/<repo> --branch main --machine basicLinux32gb`

Or to open an existing codespace:

`gh codespace ssh -c <name_of_codespace>`

- After connecting via `gh codespace ssh`, you will be inside a Linux shell that the rest of this lab assumes.

- Save screenshot as: task1_codespace_ssh_connected.png — terminal inside the Codespace shell after `gh codespace ssh -c <name>`.

```
PS C:\Users\admin\temp-repo\my-lab-repo> gh codespace create --repo Hamail-maker/my-lab-repo --machine basicLinux32gb
✓ Codespaces usage for this repository is paid for by Hamail-maker
potential-eureka-q7xw45r4wg5vhpqx
PS C:\Users\admin\temp-repo\my-lab-repo> gh codespace list
NAME              DISPLAY_NAME      REPOSITORY          BRANCH  STATE      CREATED AT
potential-eureka-q7xw45r4wg5vhpqx  potential eureka  Hamail-maker/my-lab-repo  main    Available  less than a minute ago
PS C:\Users\admin\temp-repo\my-lab-repo> |
```

Task 2 — Install AWS CLI inside the Codespace and configure it

Goal: Install the AWS CLI in the Codespace and configure it to interact with your AWS account.

Inside the Codespace shell run:

1. Download and install AWS CLI:
2. `curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"`
3. `unzip awscliv2.zip`

`sudo ./aws/install`

- Save screenshot as: task2_aws_install_and_version.png — terminal showing the download/unzip/install output.

```
@Hamail-maker → /workspaces/my-lab-repo (main) $ curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o awscliv2.zip
unzip awscliv2.zip
sudo ./aws/install
aws --version
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload  Total   Spent    Left     Speed
100 60.2M  100 60.2M    0     0  180M      0 --:--:-- --:--:-- --:--:-- 181M
Archive:  awscliv2.zip
  creating: aws/
  creating: aws/dist/
  inflating: aws/install
  inflating: aws/THIRD_PARTY_LICENSES
  inflating: aws/README.md
  creating: aws/dist/awscli/
  creating: aws/dist/dateutil/
  creating: aws/dist/docutils/
  creating: aws/dist/lib-dynload/
  creating: aws/dist/prompt_toolkit-3.0.51.dist-info/
  creating: aws/dist/wheel-0.45.1.dist-info/
  inflating: aws/dist/aws
  inflating: aws/dist/aws_completer
  inflating: aws/dist/libpython3.13.so.1.0
```

4. Verify installation:

`aws --version`

- Save screenshot as: task2_aws_install_and_version.png — include `aws --version` output (same file as install output is acceptable).

```
@Hamail-maker → /workspaces/my-lab-repo (main) $ aws --version
aws-cli/2.32.17 Python/3.13.11 Linux/6.8.0-1030-azure exe/x86_64.ubuntu.24
@Hamail-maker → /workspaces/my-lab-repo (main) $ |
```

5. Configure the AWS CLI (you will provide Access Key ID and Secret Access Key for a user with permissions, or use root/IAM user you prepared for the lab):

aws configure

- Enter:
 - AWS Access Key ID
 - AWS Secret Access Key
 - Default region name (e.g., me-central-1)
 - Default output format (e.g., json)
- Save screenshot as: task2_aws_configure_and_files.png — show aws configure prompt (redact secret values if visible).

```
@Hamail-maker → /workspaces/my-lab-repo (main) $ aws --version
aws-cli/2.32.17 Python/3.13.11 Linux/6.8.0-1030-azure exe/x86_64.ubuntu.24
@Hamail-maker → /workspaces/my-lab-repo (main) $ aws configure
AWS Access Key ID [None]: AKIA42E3QACSJXZJ35UE
AWS Secret Access Key [None]: 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
Default region name [None]: me-central-1
Default output format [None]: json
@Hamail-maker → /workspaces/my-lab-repo (main) $ |
```

6. Verify credentials/config files:

7. cat ~/.aws/credentials

cat ~/.aws/config

- Save screenshot as: task2_aws_configure_and_files.png — output of cat commands (redact secrets).

```
@Hamail-maker → /workspaces/my-lab-repo (main) $ cat ~/.aws/credentials
[default]
aws_access_key_id = AKIA42E3QACSJXZJ35UE
aws_secret_access_key = 0V+BGHfMvkckR5Hz6ztGZRIoMVqJXKeDfL/qA3V9
@Hamail-maker → /workspaces/my-lab-repo (main) $ cat ~/.aws/config
[default]
region = me-central-1
output = json
@Hamail-maker → /workspaces/my-lab-repo (main) $ |
```

8. Verify connectivity (you should see a JSON result showing your caller identity):

aws sts get-caller-identity

Expected output (example):

```
{
  "UserId": "EXAMPLEUSERID1234567890",
  "Account": "123456789012",
```

```
"Arn": "arn:aws:iam::123456789012:user/ExampleUser"
}
```

- Save screenshot as: task2_aws_get_caller_identity.png — aws sts get-caller-identity output.

```
@Hamail-maker → /workspaces/my-lab-repo (main) $ aws sts get-caller-identity
{
  "UserId": "AIDA42E3QACSNIPKW2KQG",
  "Account": "880794402980",
  "Arn": "arn:aws:iam::880794402980:user/lab-user"
}
@Hamail-maker → /workspaces/my-lab-repo (main) $ |
```

Task 3 — Create security group and add ingress rules using Codespace IP

Goal: Create an EC2 security group, inspect it, add an SSH rule for your Codespace public IP, and verify.

Steps (run in the Codespace shell):

1. Create a security group (substitute your VPC id):
2. `aws ec2 create-security-group --group-name 'MySecurityGroup' \`
3. `--description 'My Security Group' \`
`--vpc-id 'vpc-EXAMPLE1234567890'`

This command returns a security group id, for example sg-EXAMPLE1234567890.

- Save screenshot as: task3_create_security_group_output.png — output of create-security-group.

```

    }
:
  "SecurityGroups": [
    {
      "GroupId": "sg-0c69b84157c331a33",
      "IpPermissionsEgress": [
        {
          "IpProtocol": "-1",
          "UserIdGroupPairs": [],
          "IpRanges": [
            {
              "CidrIp": "0.0.0.0/0"
            }
          ]
        }
      ]
    }
  ]
}
```

4. Inspect the security group (initially ingress will be empty or default):

```
aws ec2 describe-security-groups --group-ids sg-EXAMPLE1234567890
```

- Save screenshot as: task3_describe_sg_before_ingress.png — describe-security-groups before adding rules.

```
{
  "SecurityGroups": [
    {
      "GroupId": "sg-0c69b84157c331a33",
      "IpPermissionsEgress": [
        {
          "IpProtocol": "-1",
          "UserIdGroupPairs": [],
          "IpRanges": [
            {
              "CidrIp": "0.0.0.0/0"
            }
          ],
          "Ipv6Ranges": [],
          "PrefixListIds": []
        }
      ],
      "VpcId": "vpc-02e5f1f03e6b88a38",
      "SecurityGroupArn": "arn:aws:ec2:me-central-1:880794402980:security-group/sg-0c69b84157c331a33",
      "OwnerId": "880794402980",
      "GroupName": "default",
      "Description": "default VPC security group",
      "IpPermissions": [
        {
          "IpProtocol": "-1",
          "UserIdGroupPairs": [
            {
              "UserId": "880794402980",
              "GroupId": "sg-0c69b84157c331a33"
            }
          ],
          "IpRanges": [],
          "Ipv6Ranges": [],
          "PrefixListIds": []
        }
      ]
    }
  ]
}
```

5. Get your Codespace public IP (from inside the Codespace):

`curl icanhazip.com`

- Save screenshot as: task3_codespace_public_ip.png — output of `curl icanhazip.com`.

```
@Hamail-maker → /workspaces/my-lab-repo (main) $ curl icanhazip.com
20.192.21.54
@Hamail-maker → /workspaces/my-lab-repo (main) $ |
```

6. Authorize SSH inbound on port 22 from your Codespace IP:

7. `aws ec2 authorize-security-group-ingress \`

8. `--group-id sg-EXAMPLE1234567890 \`

9. `--protocol tcp \`

10. `--port 22 \`

`--cidr <XXX.XXX.XXX.XXX>/32`

- Save screenshot as: task3_authorize_ssh_and_describe.png — `authorize-security-group-ingress` for SSH and subsequent `describe` output.

```
@Hamail-maker → /workspaces/my-lab-repo (main) $ aws ec2 authorize-security-group-ingress \
--group-id sg-0c69b84157c331a33 \
--protocol tcp \
--port 22 \
--cidr 20.192.21.54/32
```

11. Verify ingress rule appears:

`aws ec2 describe-security-groups --group-ids sg-EXAMPLE1234567890`

- Save screenshot as: task3_authorize_ssh_and_describe.png — describe output showing SSH ingress (same file as previous step is acceptable).

```
{
  "SecurityGroups": [
    {
      "GroupId": "sg-0c69b84157c331a33",
      "IpPermissionsEgress": [
        {
          "IpProtocol": "-1",
          "UserIdGroupPairs": [],
          "IpRanges": [
            {
              "CidrIp": "0.0.0.0/0"
            }
          ],
          "Ipv6Ranges": [],
          "PrefixListIds": []
        }
      ],
      "VpcId": "vpc-02e5f1f03e6b88a38",
      "SecurityGroupArn": "arn:aws:ec2:me-central-1:880794402980:security-group/sg-0c69b84157c331a33",
      "OwnerId": "880794402980",
      "GroupName": "default",
      "Description": "default VPC security group",
      "IpPermissions": [
        {
          "IpProtocol": "-1",
          "UserIdGroupPairs": [
            {
              "UserId": "880794402980",
              "GroupId": "sg-0c69b84157c331a33"
            }
          ],
          "IpRanges": [],
          "Ipv6Ranges": [],
          "PrefixListIds": []
        }
      ]
    }
  ]
}
```

12. Add an HTTP rule (port 80) using ip-permissions JSON:

13. `aws ec2 authorize-security-group-ingress \`

14. `--group-id 'sg-EXAMPLE1234567890' \`

`--ip-permissions`

`'{"FromPort":80,"ToPort":80,"IpProtocol":"tcp","IpRanges":[{"CidrIp":"<XXX.XXX.XXX.XXX >/32"}]}'`

- Save screenshot as: task3_authorize_http_and_describe.png — HTTP ip-permissions command and response.

```
@Hamail-maker →/workspaces/my-lab-repo (main) $ aws ec2 authorize-security-group-ingress \
--group-id sg-0c69b84157c331a33 \
--ip-permissions '{"FromPort":80,"ToPort":80,"IpProtocol":"tcp","IpRanges":[{"CidrIp":"20.192.21.54/32"}]}'
{
  "Return": true,
  "SecurityGroupRules": [
    {
      "SecurityGroupRuleId": "sgr-01984ca6357343bc4",
      "GroupId": "sg-0c69b84157c331a33",
      "GroupOwnerId": "880794402980",
      "IsEgress": false,
      "IpProtocol": "tcp",
      "FromPort": 80,
      "ToPort": 80,
      "CidrIpv4": "20.192.21.54/32",
      "SecurityGroupRuleArn": "arn:aws:ec2:me-central-1:880794402980:security-group-rule/sgr-01984ca6357343bc4"
    }
  ]
}
```

15. Verify both ingress rules are present:

aws ec2 describe-security-groups --group-ids sg-EXAMPLE1234567890

- Save screenshot as: task3_describe_sg_final.png — final describe showing both ingress rules.

```
@Hawaii-maker → /workspaces/my-lab-repo (main) $ aws ec2 describe-security-groups --group-ids sg-0c69b84157c331a33
{
  "SecurityGroups": [
    {
      "GroupId": "sg-0c69b84157c331a33",
      "IpPermissionsEgress": [
        {
          "IpProtocol": "-1",
          "UserIdGroupPairs": [],
          "IpRanges": [
            {
              "CidrIp": "0.0.0.0/0"
            }
          ],
          "Ipv6Ranges": [],
          "PrefixListIds": []
        }
      ],
      "VpcId": "vpc-02e5f1f03e6b88a38",
      "SecurityGroupArn": "arn:aws:ec2:me-central-1:880794402980:security-group/sg-0c69b84157c331a33",
      "OwnerId": "880794402980",
      "GroupName": "default",
      "Description": "default VPC security group",
      "IpPermissions": [
        {
          "IpProtocol": "tcp",
          "FromPort": 80,
          "ToPort": 80,
          "UserIdGroupPairs": [],
          "IpRanges": [
            {
              "CidrIp": "20.192.21.54/32"
            }
          ]
        }
      ]
    }
  ]
}
```

Task 4 — Create a key pair, describe key pairs, and launch EC2 instance

Goal: Create an ED25519 key pair, view it, and launch an EC2 instance using the key pair and the security group created earlier.

Steps:

1. Create the key pair and save the PEM file into the Codespace workspace:
 2. aws ec2 create-key-pair \
 3. --key-name MyED25519Key \
 4. --key-type ed25519 \
 5. --key-format pem \
 6. --query 'KeyMaterial' \
- output text > MyED25519Key.pem
- Save screenshot as: task4_create_keypair_output.png — output of create-key-pair and ls -l MyED25519Key.pem.

```
@Hamail-maker → /workspaces/my-lab-repo (main) $ aws ec2 create-key-pair \
--key-name MyED25519Key \
--key-type ed25519 \
--key-format pem \
--query 'KeyMaterial' \
--output text > MyED25519Key.pem

An error occurred (InvalidKeyPair.Duplicate) when calling the CreateKeyPair operation: The keypair already exists
@Hamail-maker → /workspaces/my-lab-repo (main) $ ls -l MyED25519Key.pem
-rw-rw-rw- 1 codespace codespace 0 Dec 16 11:26 MyED25519Key.pem
@Hamail-maker → /workspaces/my-lab-repo (main) $
```

7. View created key pairs:

aws ec2 describe-key-pairs

- Save screenshot as: task4_describe_keypairs.png — describe-key-pairs output.

```
@Hamail-maker → /workspaces/my-lab-repo (main) $ aws ec2 describe-key-pairs
{
  "KeyPairs": [
    {
      "KeyPairId": "key-0f6857139cf17aec6",
      "KeyType": "ed25519",
      "Tags": [],
      "CreateTime": "2025-12-16T11:07:00.825000+00:00",
      "KeyName": "MyED25519Key",
      "KeyFingerprint": "qbJc8sDOyBAGE4MM8k20AuoaxZi6sn2BrQQXk8zTdSI="
    }
  ]
}
@Hamail-maker → /workspaces/my-lab-repo (main) $ |
```

8. (Do not) Delete key pair:

aws ec2 delete-key-pair --key-name MyED25519Key # Info: shows how to delete

- Save screenshot as: task4_delete_keypair_optional.png — output of delete-key-pair (if performed).

9. Launch an EC2 instance (example values — replace IDs with ones from your account/region):

10. aws ec2 run-instances \

11. --image-id ami-05e66df2bafcb7dea \

12. --count 1 \

13. --instance-type t3.micro \

14. --key-name MyED25519Key \

15. --security-group-ids sg-EXAMPLE1234567890 \

16. --subnet-id subnet-EXAMPLE1234567890 \

--tag-specifications "ResourceType=instance,Tags=[{Key=Name,Value=MyServer}]"

- Save screenshot as: task4_run_instances_output.png — run-instances output with instance id.

```
@Hamail-maker → /workspaces/my-lab-repo (main) $ aws ec2 run-instances \
--image-id ami-05e66df2bafcb7dea \
--count 1 \
--instance-type t3.micro \
--key-name MyED25519Key \
--security-group-ids sg-0c69b84157c331a33 \
--subnet-id subnet-0695f75f71a335d1f \
--tag-specifications "ResourceType=instance,Tags=[{Key=Name,Value=MyServer}]"
{
  "ReservationId": "r-020b21262751ef28b",
  "OwnerId": "880794402980",
  "Groups": [],
  "Instances": [
    {
      "Architecture": "x86_64",
      "BlockDeviceMappings": [],
      "ClientToken": "79fb6689-5be2-49d5-9a43-126e90aefaec",
      "EbsOptimized": false,
      "EnaSupport": true,
      "Hypervisor": "xen",
      "NetworkInterfaces": [
        {
          "Attachment": {
            "AttachTime": "2025-12-16T13:22:14+00:00",
            "AttachmentId": "eni-attach-083c2ab59f3e67e0d",
            "DeleteOnTermination": true,
            "DeviceIndex": 0,
            "Status": "attaching",
            "NetworkCardIndex": 0
          },
          "Description": "",
          "Groups": [
```

17. Get the public IP address of your instance:

18. `aws ec2 describe-instances \`

19. `--query "Reservations[*].Instances[*].[InstanceId,PublicIpAddress]" \`

`--output table`

- Save screenshot as: task4_describe_instances_public_ip.png — describe-instances table with public IP.

```
@Hamail-maker → /workspaces/my-lab-repo (main) $ aws ec2 describe-instances \
--query "Reservations[*].Instances[*].[InstanceId,PublicIpAddress]" \
--output table
-----
|          DescribeInstances          |
+-----+-----+
| i-0d311ef7e89b9f26a | 3.28.204.157 |
+-----+-----+
@Hamail-maker → /workspaces/my-lab-repo (main) $ |
```

20. Attempt SSH into the instance from the Codespace or from a machine whose IP is allowed in the security group:

`ssh -i MyED25519Key.pem ec2-user@<public-ip-address>`

- If you see the error:
- Permissions 0644 for 'MyED25519Key.pem' are too open. It is required that your private key files are NOT accessible by others.

fix permissions:

chmod 400 MyED25519Key.pem

ssh -i MyED25519Key.pem ec2-user@<public-ip-address>

- Save screenshot as: task4_ssh_permission_error_and_fix.png — show permission error and chmod 400 then successful SSH (redact sensitive details).

```
@Hamail-maker → /workspaces/my-lab-repo (main) $ ssh -i MyED25519Key.pem ec2-user@3.28.204.157
ssh: connect to host 3.28.204.157 port 22: Connection timed out
@Hamail-maker → /workspaces/my-lab-repo (main) $
@Hamail-maker → /workspaces/my-lab-repo (main) $ ssh -i MyED25519Key.pem ec2-user@3.28.204.157
ssh: connect to host 3.28.204.157 port 22: Connection timed out
@Hamail-maker → /workspaces/my-lab-repo (main) $ chmod 400 MyED25519Key.pem
@Hamail-maker → /workspaces/my-lab-repo (main) $ ssh -i MyED25519Key.pem ec2-user@3.28.204.157
ssh: connect to host 3.28.204.157 port 22: Connection timed out
@Hamail-maker → /workspaces/my-lab-repo (main) $ |
```

21. Stop, start and (optionally) terminate the instance:

22. aws ec2 stop-instances --instance-ids i-EXAMPLE1234567890

23. aws ec2 start-instances --instance-ids i-EXAMPLE1234567890

aws ec2 terminate-instances --instance-ids i-EXAMPLE1234567890 # Don't run this command

- Save screenshot as: task4_stop_start_terminate_commands.png — outputs for stop/start/terminate.

```
@Hamail-maker → /workspaces/my-lab-repo (main) $ aws ec2 stop-instances --instance-ids i-0d311ef7e89b9f26a
{
  "StoppingInstances": [
    {
      "InstanceId": "i-0d311ef7e89b9f26a",
      "CurrentState": {
        "Code": 80,
        "Name": "stopped"
      },
      "PreviousState": {
        "Code": 80,
        "Name": "stopped"
      }
    }
  ]
}
@Hamail-maker → /workspaces/my-lab-repo (main) $ aws ec2 start-instances --instance-ids i-0d311ef7e89b9f26a
{
  "StartingInstances": [
    {
      "InstanceId": "i-0d311ef7e89b9f26a",
      "CurrentState": {
        "Code": 0,
        "Name": "pending"
      },
      "PreviousState": {
        "Code": 80,
        "Name": "stopped"
      }
    }
  ]
}
@Hamail-maker → /workspaces/my-lab-repo (main) $ aws ec2 terminate-instances --instance-ids i-0d311ef7e89b9f26a|
```

Task 5 — Understand AWS describe-* commands

Goal: Use describe commands to list and inspect AWS resources.

Run and understand these commands (run each, then capture screenshot immediately after):

aws ec2 describe-security-groups

- Save screenshot as: task5_describe_security_groups.png — output of describe-security-groups.

```
@Hamaill-maker → /workspaces/my-lab-repo (main) $ aws ec2 describe-security-groups
{
  "SecurityGroups": [
    {
      "GroupId": "sg-0c69b84157c331a33",
      "IpPermissionsEgress": [
        {
          "IpProtocol": "-1",
          "UserIdGroupPairs": [],
          "IpRanges": [
            {
              "CidrIp": "0.0.0.0/0"
            }
          ],
          "Ipv6Ranges": [],
          "PrefixListIds": []
        }
      ],
      "VpcId": "vpc-02e5f1f03e6b88a38",
      "SecurityGroupArn": "arn:aws:ec2:me-central-1:880794402980:security-group/sg-0c69b84157c331a33",
      "OwnerId": "880794402980",
      "GroupName": "default",
      "Description": "default VPC security group",
    }
  ]
}
```

aws ec2 describe-vpcs

- Save screenshot as: task5_describe_vpcs.png — output of describe-vpcs.

aws ec2 describe-subnets

```
@Hamaill-maker → /workspaces/my-lab-repo (main) $ aws ec2 describe-vpcs
{
  "Vpcs": [
    {
      "OwnerId": "880794402980",
      "InstanceTenancy": "default",
      "CidrBlockAssociationSet": [
        {
          "AssociationId": "vpc-cidr-assoc-0f7f2af88f5d889fa",
          "CidrBlock": "172.31.0.0/16",
          "CidrBlockState": {
            "State": "associated"
          }
        }
      ],
      "IsDefault": true,
      "BlockPublicAccessStates": {
        "InternetGatewayBlockMode": "off"
      },
      "VpcId": "vpc-02e5f1f03e6b88a38",
      "State": "available",
      "CidrBlock": "172.31.0.0/16",
      "DhcpOptionsId": "dopt-00cbdc96c2de8964b"
    }
  ]
}
```

- Save screenshot as: task5_describe_subnets.png — output of describe-subnets.

aws ec2 describe-subnets

```
@Hamail-maker → /workspaces/my-lab-repo (main) $ aws ec2 describe-subnets
{
  "Subnets": [
    {
      "AvailabilityZoneId": "mec1-az1",
      "MapCustomerOwnedIpOnLaunch": false,
      "OwnerId": "880794402980",
      "AssignIpv6AddressOnCreation": false,
      "Ipv6CidrBlockAssociationSet": [],
      "SubnetArn": "arn:aws:ec2:me-central-1:880794402980:subnet/subnet-0695f75f71a335d1f",
      "EnableDns64": false,
      "Ipv6Native": false,
      "PrivateDnsNameOptionsOnLaunch": {
        "HostnameType": "ip-name",
        "EnableResourceNameDnsARecord": false,
        "EnableResourceNameDnsAAAARecord": false
      },
      "BlockPublicAccessStates": {
        "InternetGatewayBlockMode": "off"
      },
      "SubnetId": "subnet-0695f75f71a335d1f",
      "State": "available",
      "VpcId": "vpc-02e5f1f03e6b88a38",
      "CidrBlock": "172.31.32.0/20",
      "AvailableIpAddressCount": 4090,
      "AvailabilityZone": "me-central-1a",
      "DefaultForAz": true,
      "MapPublicIpOnLaunch": true
    },
    {
      "AvailabilityZoneId": "mec1-az3",
      "MapCustomerOwnedIpOnLaunch": false,
      "OwnerId": "880794402980",
```

- Save screenshot as: task5_describe_instances.png — output of describe-instances.

aws ec2 describe-instances

```
@Hamail-maker → /workspaces/my-lab-repo (main) $ aws ec2 describe-instances
{
  "Reservations": [
    {
      "ReservationId": "r-020b21262751ef28b",
      "OwnerId": "880794402980",
      "Groups": [],
      "Instances": [
        {
          "Architecture": "x86_64",
          "BlockDeviceMappings": [
            {
              "DeviceName": "/dev/xvda",
              "Ebs": {
                "AttachTime": "2025-12-16T13:22:15+00:00",
                "DeleteOnTermination": true,
                "Status": "attached",
                "VolumeId": "vol-0895adbbbeaaf75c9"
              }
            }
          ]
        }
      ]
    }
  ],
```

- Save screenshot as: task5_describe_regions.png — output of describe-regions.

aws ec2 describe-availability-zones

```
@Hamail-maker → /workspaces/my-lab-repo (main) $ aws ec2 describe-regions
{
  "Regions": [
    {
      "OptInStatus": "opt-in-not-required",
      "RegionName": "ap-south-1",
      "Endpoint": "ec2.ap-south-1.amazonaws.com"
    },
    {
      "OptInStatus": "opt-in-not-required",
      "RegionName": "eu-north-1",
      "Endpoint": "ec2.eu-north-1.amazonaws.com"
    },
    {
      "OptInStatus": "opt-in-not-required",
      "RegionName": "eu-west-3",
      "Endpoint": "ec2.eu-west-3.amazonaws.com"
    },
    {
      "OptInStatus": "opt-in-not-required",
      "RegionName": "eu-west-2",
      "Endpoint": "ec2.eu-west-2.amazonaws.com"
    },
    {
      "OptInStatus": "opt-in-not-required",
      "RegionName": "eu-west-1",
      "Endpoint": "ec2.eu-west-1.amazonaws.com"
    }
  ]
}
```

- Save screenshot as: task5_describe_availability_zones.png — output of describe-availability-zones.

```
@Hamail-maker → /workspaces/my-lab-repo (main) $ aws ec2 describe-availability-zones
{
  "AvailabilityZones": [
    {
      "OptInStatus": "opt-in-not-required",
      "Messages": [],
      "RegionName": "me-central-1",
      "ZoneName": "me-central-1a",
      "ZoneId": "mec1-az1",
      "GroupName": "me-central-1-zg-1",
      "NetworkBorderGroup": "me-central-1",
      "ZoneType": "availability-zone",
      "GroupLongName": "Middle East (UAE) 1",
      "State": "available"
    },
    {
      "OptInStatus": "opt-in-not-required",
      "Messages": [],
      "RegionName": "me-central-1",
      "ZoneName": "me-central-1b",
      "ZoneId": "mec1-az2",
      "GroupName": "me-central-1-zg-1",
      "NetworkBorderGroup": "me-central-1",
      "ZoneType": "availability-zone",
      "GroupLongName": "Middle East (UAE) 1",
      "State": "available"
    },
    {
      "OptInStatus": "opt-in-not-required",
      "Messages": [],
      "RegionName": "me-central-1",
      "ZoneName": "me-central-1c",
      "ZoneId": "mec1-az3",
      "GroupName": "me-central-1-zg-1",
      "NetworkBorderGroup": "me-central-1",
      "ZoneType": "availability-zone",
      "GroupLongName": "Middle East (UAE) 1",
      "State": "available"
    }
  ]
}
```

Task 6 — IAM: create group, user, attach policies, create console login & keys

Goal: Practice managing IAM via CLI: create group and user, attach policies, add user to group, create login profile, attach/detach change-password policy, create access keys, and test access via environment variables.

Commands and immediate screenshot request after each step:

1. Create group:

aws iam create-group --group-name MyGroupCli

- Save screenshot as: task6_create_group_and_user.png — create-group output.

```
@Hamail-maker → /workspaces/my-lab-repo (main) $ aws iam create-group --group-name MyGroupCli
{
  "Group": {
    "Path": "/",
    "GroupName": "MyGroupCli",
    "GroupId": "AGPA42E3QACSNBIV5NV5I",
    "Arn": "arn:aws:iam::880794402980:group/MyGroupCli",
    "CreateDate": "2025-12-16T13:52:27+00:00"
  }
}
```

2. Get group details:

aws iam get-group --group-name MyGroupCli

- Save screenshot as: task6_create_group_and_user.png — get-group output.

```
@Hamail-maker → /workspaces/my-lab-repo (main) $ aws iam get-group --group-name MyGroupCli
{
  "Users": [],
  "Group": {
    "Path": "/",
    "GroupName": "MyGroupCli",
    "GroupId": "AGPA42E3QACSNBIV5NV5I",
    "Arn": "arn:aws:iam::880794402980:group/MyGroupCli",
    "CreateDate": "2025-12-16T13:52:27+00:00"
  }
}
```

3. Create user:

aws iam create-user --user-name MyUserCli

- Save screenshot as: task6_create_group_and_user.png — create-user output.

```
@Hamail-maker → /workspaces/my-lab-repo (main) $ aws iam create-user --user-name MyUserCli
{
  "User": {
    "Path": "/",
    "UserName": "MyUserCli",
    "UserId": "AIDA42E3QACS05XH5NPBG",
    "Arn": "arn:aws:iam::880794402980:user/MyUserCli",
    "CreateDate": "2025-12-16T13:54:12+00:00"
  }
}
```


4. Get user details:

aws iam get-user --user-name MyUserCli

- Save screenshot as: task6_create_group_and_user.png — get-user output.

```
@Hamail-maker → /workspaces/my-lab-repo (main) $ aws iam get-user --user-name MyUserCli
{
  "User": {
    "Path": "/",
    "UserName": "MyUserCli",
    "UserId": "AIDA42E3QACS05XH5NPBG",
    "Arn": "arn:aws:iam::880794402980:user/MyUserCli",
    "CreateDate": "2025-12-16T13:54:12+00:00"
  }
}
```

4. Add user to group:

aws iam add-user-to-group --user-name MyUserCli --group-name MyGroupCli

- Save screenshot as: task6_add_user_to_group_and_verify.png — add-user-to-group and verify with get-group.

```
@Hamail-maker → /workspaces/my-lab-repo (main) $ aws iam add-user-to-group \
--user-name MyUserCli \
--group-name MyGroupCli
```

5. See group again:

aws iam get-group --group-name MyGroupCli

- Save screenshot as: task6_add_user_to_group_and_verify.png — get-group showing user present.

```
@Hamail-maker → /workspaces/my-lab-repo (main) $ aws iam get-group --group-name MyGroupCli
{
  "Users": [
    {
      "Path": "/",
      "UserName": "MyUserCli",
      "UserId": "AIDA42E3QACS05XH5NPBG",
      "Arn": "arn:aws:iam::880794402980:user/MyUserCli",
      "CreateDate": "2025-12-16T13:54:12+00:00"
    }
  ],
  "Group": {
    "Path": "/",
    "GroupName": "MyGroupCli",
    "GroupId": "AGPA42E3QACSNBIV5NV5I",
    "Arn": "arn:aws:iam::880794402980:group/MyGroupCli",
    "CreateDate": "2025-12-16T13:52:27+00:00"
  }
}
```

6. List policies that mention EC2:

7. aws iam list-policies \

8. --query "Policies[?contains(PolicyName, 'EC2')].{Name:PolicyName}" \

--output text

- Save screenshot as: task6_policy_list_and_attach.png — policy list output.

```
@Hamail-maker → /workspaces/my-lab-repo (main) $ aws iam list-policies \
  --query "Policies[?contains(PolicyName, 'EC2')].{Name:PolicyName}" \
  --output text
AmazonEC2FullAccess
AmazonEC2ReadOnlyAccess
AmazonElasticMapReduceforEC2Role
AmazonEC2RoleforDataPipelineRole
AmazonEC2ContainerServiceforEC2Role
AmazonEC2ContainerServiceRole
AmazonEC2RoleforAWSCodeDeploy
AmazonEC2RoleforSSM
CloudWatchActionsEC2Access
AmazonEC2ContainerRegistryReadOnly
AmazonEC2ContainerRegistryPowerUser
AmazonEC2ContainerRegistryFullAccess
AmazonEC2ContainerServiceAutoscaleRole
AmazonEC2SpotFleetAutoscaleRole
AWSElasticBeanstalkCustomPlatformforEC2Role
AmazonEC2ContainerServiceEventsRole
AmazonEC2SpotFleetTaggingRole
AWSEC2SpotServiceRolePolicy
AWSServiceRoleForEC2ScheduledInstances
AWSEC2SpotFleetServiceRolePolicy
AWSApplicationAutoscalingEC2SpotFleetRequestPolicy
AWSEC2FleetServiceRolePolicy
AWSAutoScalingPlansEC2AutoScalingPolicy
```

9. Get ARN for AmazonEC2FullAccess (example query):

```
aws iam list-policies --query
'Policies[?PolicyName=='AmazonEC2FullAccess'].{Name:PolicyName, ARN:Arn}' --output
table
```

- Save screenshot as: task6_policy_list_and_attach.png — ARN query output.

ListPolicies	
ARN	Name
arn:aws:iam::aws:policy/AmazonEC2FullAccess	AmazonEC2FullAccess

10. Attach policy to group (use the ARN you retrieved):

11. aws iam attach-group-policy \

12. --group-name MyGroupCli \

```
--policy-arn arn:aws:iam::aws:policy/EXAMPLEPolicyName
```

- Save screenshot as: task6_policy_list_and_attach.png — attach-group-policy output.

```
@Hamail-maker →/workspaces/my-lab-repo (main) $ aws iam attach-group-policy \
--group-name MyGroupCli \
--policy-arn arn:aws:iam::aws:policy/AmazonEC2FullAccess
@Hamail-maker →/workspaces/my-lab-repo (main) $ |
```

13. List attached policies for the group:

```
aws iam list-attached-group-policies --group-name MyGroupCli
```

- Save screenshot as: task6_policy_list_and_attach.png — list-attached-group-policies output.

```
@Hamail-maker →/workspaces/my-lab-repo (main) $ aws iam list-attached-group-policies --group-name MyGroupCli
{
  "AttachedPolicies": [
    {
      "PolicyName": "AmazonEC2FullAccess",
      "PolicyArn": "arn:aws:iam::aws:policy/AmazonEC2FullAccess"
    }
  ]
}
@Hamail-maker →/workspaces/my-lab-repo (main) $ |
```

14. Create a console login profile for the user:

15. aws iam create-login-profile \

16. --user-name MyUserCli \

17. --password <PASSWORD_VALUE> \

```
--password-reset-required
```

- Save screenshot as: task6_create_login_profile_and_signin.png — create-login-profile output (do not show password).

```
@Hamail-maker →/workspaces/my-lab-repo (main) $ aws iam create-login-profile \
--user-name MyUserCli \
--password Temp@1234 \
--password-reset-required
{
  "LoginProfile": {
    "UserName": "MyUserCli",
    "CreateDate": "2025-12-16T14:00:34+00:00",
    "PasswordResetRequired": true
  }
}
```

18. If the user cannot change password, attach IAMUserChangePassword to the group temporarily:

```
aws iam attach-group-policy --group-name MyGroupCli --policy-arn
arn:aws:iam::aws:policy/IAMUserChangePassword
```

After the user logs in and resets password, detach that policy:

```
aws iam detach-group-policy --group-name MyGroupCli --policy-arn
arn:aws:iam::aws:policy/IAMUserChangePassword
```

- Save screenshot as: task6_create_login_profile_and_signin.png — attach/detach outputs and a screenshot showing the user login (redact password).

```
@Hamail-maker → /workspaces/my-lab-repo (main) $ aws iam attach-group-policy \
--group-name MyGroupCli \
--policy-arn arn:aws:iam::aws:policy/IAMUserChangePassword
@Hamail-maker → /workspaces/my-lab-repo (main) $ aws iam detach-group-policy \
--group-name MyGroupCli \
--policy-arn arn:aws:iam::aws:policy/IAMUserChangePassword
@Hamail-maker → /workspaces/my-lab-repo (main) $ |
```

19. Create access keys for the user (save AccessKeyId and SecretAccessKey securely):

```
aws iam create-access-key --user-name MyUserCli
```

- Save screenshot as: task6_create_access_key_output.png — create-access-key output (redact keys).

```
@Hamail-maker → /workspaces/my-lab-repo (main) $ aws iam create-access-key --user-name MyUserCli
{
  "AccessKey": {
    "UserName": "MyUserCli",
    "AccessKeyId": "AKIA42E3QACSM7NT6GA",
    "Status": "Active",
    "SecretAccessKey": "0mOX6v0/Ff2/Z30KQFJh/G6uLqf96i+40ubyewWv",
    "CreateDate": "2025-12-16T14:02:21+00:00"
  }
}
@Hamail-maker → /workspaces/my-lab-repo (main) $ |
```

20. List access keys:

```
aws iam list-access-keys --user-name MyUserCli
```

- Save screenshot as: task6_create_access_key_output.png — list-access-keys output.

```
@Hamail-maker → /workspaces/my-lab-repo (main) $ aws iam list-access-keys --user-name MyUserCli
{
  "AccessKeyMetadata": [
    {
      "UserName": "MyUserCli",
      "AccessKeyId": "AKIA42E3QACSM7NT6GA",
      "Status": "Active",
      "CreateDate": "2025-12-16T14:02:21+00:00"
    }
  ]
}
```

21. (Don't) Delete access key:

```
aws iam delete-access-key --user-name MyUserCli --access-key-id <AccessKeyId> # Don't
run this command
```

- Save screenshot as: task6_create_access_key_output.png — delete-access-key output (if performed).

```
@Hamail-maker → /workspaces/my-lab-repo (main) $ export AWS_ACCESS_KEY_ID=TESTKEY123
export AWS_SECRET_ACCESS_KEY=TESTSECRET123
printenv | grep AWS_
aws iam get-user --user-name MyUserCli
AWS_SECRET_ACCESS_KEY=TESTSECRET123
AWS_ACCESS_KEY_ID=TESTKEY123

An error occurred (InvalidClientTokenId) when calling the GetUser operation: The security token included in the request is invalid.
```

22. Use environment variables to authenticate as that user in the Codespace:

23. export AWS_ACCESS_KEY_ID=<YOUR_ACCESS_KEY_ID>

24. export AWS_SECRET_ACCESS_KEY=<YOUR_SECRET_ACCESS_KEY>

25. printenv | grep AWS_

26. aws iam get-user --user-name MyUserCli # may fail if no permissions

exit # to clear exports

- Save screenshot as: task6_env_exports_and_get_user_error.png — show env exports and any AccessDenied error (if occurs).

```
An error occurred (InvalidClientTokenId) when calling the CreateAccessKey operation: The security token included in the request is invalid.
@Hamail-maker → /workspaces/my-lab-repo (main) $ aws iam get-user --user-name MyUserCli

An error occurred (InvalidClientTokenId) when calling the GetUser operation: The security token included in the request is invalid.
@Hamail-maker → /workspaces/my-lab-repo (main) $ |
```

- After clearing or switching credentials, repeat get-user and save:
- Save screenshot as: task6_after_logout_and_get_user_success.png — successful get-user output under appropriate credentials.

```
@Hamail-maker → /workspaces/my-lab-repo (main) $ unset AWS_ACCESS_KEY_ID
unset AWS_SECRET_ACCESS_KEY
@Hamail-maker → /workspaces/my-lab-repo (main) $ aws iam get-user --user-name MyUserCli
{
  "User": {
    "Path": "/",
    "UserName": "MyUserCli",
    "UserId": "AIDA42E3QACS05XH5NPBG",
    "Arn": "arn:aws:iam::880794402980:user/MyUserCli",
    "CreateDate": "2025-12-16T13:54:12+00:00"
  }
}
```

Task 7 — Filters: query with filters to find instances and their attributes

Goal: Use filters and queries to list specific instances and attributes.

Examples (run each and take a screenshot immediately after):

aws ec2 describe-instances \

--filters "Name=tag:Name,Values=MyServer" \

--query "Reservations[*].Instances[*].PublicIpAddress" \

--output text

- Save screenshot as: task7_filter_by_tag_public_ip.png — output of the filter by tag showing public IP.

```
@Hamaill-maker → /workspaces/my-lab-repo (main) $ aws ec2 describe-instances \
  --filters "Name=tag:Name,Values=MyServer" \
  --query "Reservations[*].Instances[*].PublicIpAddress" \
  --output text
158.252.95.32
@Hamaill-maker → /workspaces/my-lab-repo (main) $ |
```

```
aws ec2 describe-instances \
```

```
--filters "Name=instance-type,Values=t3.micro" \
```

```
--query "Reservations[].Instances[].InstanceId" \
```

```
--output table
```

- Save screenshot as: task7_filter_by_instance_type.png — output listing instance IDs.

```
@Hamaill-maker → /workspaces/my-lab-repo (main) $ aws ec2 describe-instances \
  --filters "Name=instance-type,Values=t3.micro" \
  --query "Reservations[].Instances[].InstanceId" \
  --output table
-----
| DescribeInstances |
+-----+
| i-0d311ef7e89b9f26a |
+-----+
@Hamaill-maker → /workspaces/my-lab-repo (main) $ |
```

```
aws ec2 describe-instances \
```

```
--filters "Name=subnet-id,Values=subnet-0600df5fa8ce60857" \
```

```
--query "Reservations[*].Instances[*].InstanceId" \
```

```
--output table
```

- Save screenshot as: task7_filter_by_subnet.png — output for subnet filter.

```
@Hamaill-maker → /workspaces/my-lab-repo (main) $ aws ec2 describe-instances \
  --filters "Name=subnet-id,Values=subnet-0695f75f71a335d1f" \
  --query "Reservations[*].Instances[*].InstanceId" \
  --output table
-----
| DescribeInstances |
+-----+
| i-0d311ef7e89b9f26a |
+-----+
@Hamaill-maker → /workspaces/my-lab-repo (main) $ |
```

```
aws ec2 describe-instances \
```

```
--filters "Name=vpc-id,Values=vpc-06be85cd81b657192" \
```

```
--query "Reservations[*].Instances[*].InstanceId" \  
  
--output table
```

- Save screenshot as: task7_filter_by_vpc.png — output for VPC filter.

```
@Hamail-maker → /workspaces/my-lab-repo (main) $ aws ec2 describe-instances \  
  --filters "Name=vpc-id,Values=vpc-02e5f1f03e6b88a38" \  
  --query "Reservations[*].Instances[*].InstanceId" \  
  --output table  
-----  
| DescribeInstances |  
+-----+  
| i-0d311ef7e89b9f26a |  
+-----+  
@Hamail-maker → /workspaces/my-lab-repo (main) $ |
```

Task 8 — Use --query to format outputs for reporting

Goal: Extract useful fields in table format for reporting.

Examples (run each and take a screenshot immediately after):

```
aws ec2 describe-instances \  
  
  --filters "Name=tag:Name,Values=MyServer" \  
  
  --query  
"Reservations[*].Instances[*].[InstanceId,PublicIpAddress,Tags[?Key=='Name'].Value|[0]]"  
\  
  
  --output table
```

- Save screenshot as: task8_query_table_instances_name_ip.png — table showing InstanceId, PublicIpAddress, Name.

```
@Hamail-maker → /workspaces/my-lab-repo (main) $ aws ec2 describe-instances \  
  --filters "Name=tag:Name,Values=MyServer" \  
  --query "Reservations[*].Instances[*].[InstanceId,PublicIpAddress,Tags[?Key=='Name'].Value|[0]]" \  
  --output table  
-----  
| InstanceId | PublicIpAddress | Name |  
+-----+-----+-----+  
| i-0d311ef7e89b9f26a | 158.252.95.32 | MyServer |  
+-----+-----+-----+  
@Hamail-maker → /workspaces/my-lab-repo (main) $ |
```

```
aws ec2 describe-instances \  
  
  --query "Reservations[*].Instances[*].[InstanceId,State.Name]" \  
  
  --output table
```

- Save screenshot as: task8_query_table_instance_state.png — table showing InstanceId and State.

```
@Hamaill-maker → /workspaces/my-lab-repo (main) $ aws ec2 describe-instances \
--query "Reservations[*].Instances[*].[InstanceId,State.Name]" \
--output table
-----
| DescribeInstances |
+-----+-----+
| i-0d311ef7e89b9f26a | running |
+-----+-----+
@Hamaill-maker → /workspaces/my-lab-repo (main) $ |
```

aws ec2 describe-instances \

--query

"Reservations[*].Instances[*].[InstanceId,InstanceType,Placement.AvailabilityZone]" \

--output table

- Save screenshot as: task8_query_table_instance_type_az.png — table showing InstanceId, InstanceType, AvailabilityZone.

```
@Hamaill-maker → /workspaces/my-lab-repo (main) $ aws ec2 describe-instances \
--query "Reservations[*].Instances[*].[InstanceId,InstanceType,Placement.AvailabilityZone]" \
--output table
-----
| DescribeInstances |
+-----+-----+
| i-0d311ef7e89b9f26a | t3.micro | me-central-1a |
+-----+-----+
@Hamaill-maker → /workspaces/my-lab-repo (main) $ |
```

Cleanup — Remove resources to avoid charges

After verification, terminate and delete everything you created in AWS. Capture screenshots for each step.

Cleanup steps and required screenshots (take each screenshot immediately after running the command):

1. Terminate instances:

aws ec2 terminate-instances --instance-ids i-EXAMPLE1234567890

- Save screenshot as: cleanup_terminate_instance.png — terminate-instances output/confirmation.

```
@Hamaill-maker → /workspaces/my-lab-repo (main) $ aws ec2 terminate-instances --instance-ids i-0d311ef7e89b9f26a
{
  "TerminatingInstances": [
    {
      "InstanceId": "i-0d311ef7e89b9f26a",
      "CurrentState": {
        "Code": 32,
        "Name": "shutting-down"
      },
      "PreviousState": {
        "Code": 16,
        "Name": "running"
      }
    }
  ]
}
```

2. Delete EBS volumes & snapshots (if any):

- Save screenshot as: cleanup_delete_volumes_snapshots.png — confirmation or listing showing volumes/snapshots deleted.

```
@Hamail-maker → /workspaces/my-lab-repo (main) $ aws ec2 describe-volumes --output table
```

DescribeVolumes	
Volumes	
AvailabilityZone	me-central-1a
AvailabilityZoneId	mec1-az1
CreateTime	2025-12-16T13:22:15.317000+00:00
Encrypted	False
Iops	3000
MultiAttachEnabled	False
Size	8
SnapshotId	snap-041b493405a2d89fb
State	in-use
Throughput	125
VolumeId	vol-0895adbbbeaaf75c9
VolumeType	gp3
Attachments	
AttachTime	2025-12-16T13:22:15+00:00
DeleteOnTermination	True
Device	/dev/xvda
InstanceId	i-0d311ef7e89b9f26a
State	attached
VolumeId	vol-0895adbbbeaaf75c9
Operator	
Managed	False

```
@Hamail-maker → /workspaces/my-lab-repo (main) $ |
```

3. Delete security group and key pair:
4. `aws ec2 delete-security-group --group-id sg-EXAMPLE1234567890`
`aws ec2 delete-key-pair --key-name MyED25519Key`

- Save screenshot as: cleanup_delete_security_group_and_keypair.png — deletion confirmation(s).

```
@Hamail-maker → /workspaces/my-lab-repo (main) $ aws ec2 delete-key-pair --key-name MyED25519Key
{
  "Return": true,
  "KeyPairId": "key-0f6857139cf17aec6"
}
@Hamail-maker → /workspaces/my-lab-repo (main) $ |
```

5. Remove IAM users, access keys, groups:
6. `aws iam delete-access-key --user-name MyUserCli --access-key-id <AccessKeyId>`
7. `aws iam delete-login-profile --user-name MyUserCli`
8. `aws iam remove-user-from-group --user-name MyUserCli --group-name MyGroupCli`
9. `aws iam delete-user --user-name MyUserCli`

10. `aws iam detach-group-policy --group-name MyGroupCli --policy-arn arn:aws:iam::aws:policy/EXAMPLEPolicyName`

`aws iam delete-group --group-name MyGroupCli`

- Save screenshot as: `cleanup_iam_users_deleted.png` — IAM deletion commands and confirmation.

```
@Hamaill-maker → /workspaces/my-lab-repo (main) $ aws iam delete-access-key \
--user-name MyUserCli \
--access-key-id AIDA42E3QACS05XH5NPBG

An error occurred (NoSuchEntity) when calling the DeleteAccessKey operation: The Access Key with id AIDA42E3QACS05XH5NPBG cannot be found.

An error occurred (CannotDelete) when calling the DeleteSecurityGroup operation: the specified group: "sg-0c69b84157c331a33" name: "default" cannot be deleted by a user
@Hamaill-maker → /workspaces/my-lab-repo (main) $ aws ec2 delete-key-pair --key-name MyED25519Key
{
  "Return": true,
  "KeyPairId": "key-0f6857139cf17aec6"
}
@Hamaill-maker → /workspaces/my-lab-repo (main) $ aws iam delete-access-key \
--user-name MyUserCli \
--access-key-id <AccessKeyId>
-bash: syntax error near unexpected token `newline'
@Hamaill-maker → /workspaces/my-lab-repo (main) $ aws iam delete-access-key \
--user-name MyUserCli \
--access-key-id AIDA42E3QACS05XH5NPBG

An error occurred (NoSuchEntity) when calling the DeleteAccessKey operation: The Access Key with id AIDA42E3QACS05XH5NPBG cannot be found.
@Hamaill-maker → /workspaces/my-lab-repo (main) $ aws iam delete-login-profile --user-name MyUserCli
@Hamaill-maker → /workspaces/my-lab-repo (main) $ aws iam delete-login-profile --user-name MyUserCli

An error occurred (NoSuchEntity) when calling the DeleteLoginProfile operation: Login Profile for User MyUserCli cannot be found.
@Hamaill-maker → /workspaces/my-lab-repo (main) $ aws iam detach-group-policy \
--group-name MyGroupCli \
--policy-arn arn:aws:iam::aws:policy/AmazonEC2FullAccess
@Hamaill-maker → /workspaces/my-lab-repo (main) $ aws iam delete-user --user-name MyUserCli

An error occurred (DeleteConflict) when calling the DeleteUser operation: Cannot delete entity, must remove users from group first.
@Hamaill-maker → /workspaces/my-lab-repo (main) $ aws iam delete-group --group-name MyGroupCli

An error occurred (DeleteConflict) when calling the DeleteGroup operation: Cannot delete entity, must remove users from group first.
@Hamaill-maker → /workspaces/my-lab-repo (main) $
```

11. Final verification (billing/resource groups):

- Save screenshot as: `cleanup_summary.png` — final console verification (billing/resource groups) showing no active resources if possible.

DescribeInstances	
Reservations	
OwnerId	880794402980
ReservationId	r-020b21262751ef28b
Instances	
AmiLaunchIndex	0
Architecture	x86_64
BootMode	uefi-preferred
ClientToken	79fb6689-5be2-49d5-9a43-126e90aefaec
CurrentInstanceBootMode	uefi
EbsOptimized	False
EnaSupport	True
Hypervisor	xen
ImageId	ami-05e66df2bafcb7dea
InstanceId	i-0d311ef7e89b9f26a
InstanceType	t3.micro
KeyName	MyED25519Key
LaunchTime	2025-12-16T13:39:21+00:00
PlatformDetails	Linux/UNIX
PrivateDnsName	
PublicDnsName	
RootDeviceName	/dev/xvda
RootDeviceType	ebs
StateTransitionReason	User initiated (2025-12-16 14:19:53 GMT)
UsageOperation	RunInstances
UsageOperationUpdateTime	2025-12-16T13:22:14+00:00
VirtualizationType	hvm
CapacityReservationSpecification	